



UNIVERSITÀ DEGLI STUDI DI MILANO

Sicurezza & Privacy

Laboratorio 1: Linux

Andrea Monzani
Prof. Danilo Bruschi





Il sistema operativo Linux



UNIX a brief Intro (<http://www.levenez.com/unix/>)

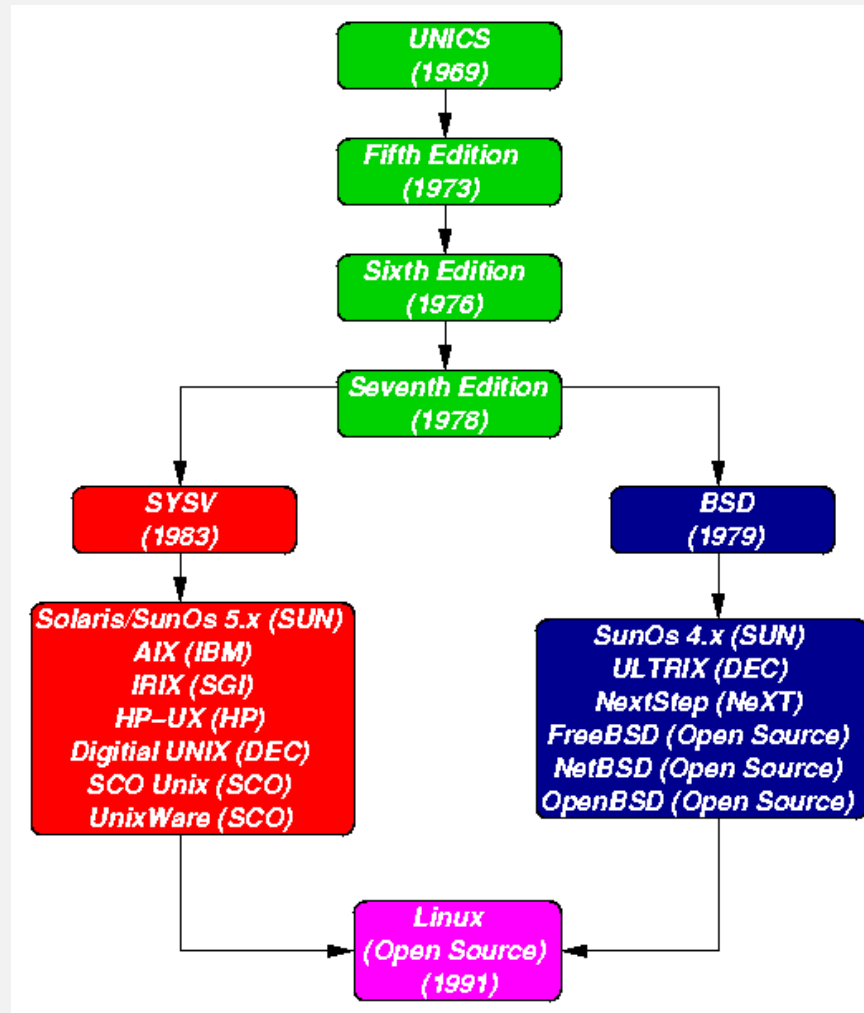
- UNIX has been a popular OS for more than two decades because of its multi-user, multi-tasking environment, stability, portability and powerful networking capabilities
- In the late 1960s, researchers from General Electric, MIT and Bell Labs launched a joint project to develop an ambitious multi-user, multi-tasking OS for mainframe computers known as MULTICS (Multiplexed Information and Computing System)
- MULTICS failed (for [some MULTICS enthusiasts](#) "failed" is perhaps too strong a word to use here), but it did inspire Ken Thompson, who was a researcher at Bell Labs, to have a go at writing a simpler operating system himself



Unix a brief Introduction

- He wrote a simpler version of MULTICS on a PDP7 in assembler and called his attempt UNICS (Uniplexed Information and Computing System) , eventually shortened to UNIX
- Ken Thompson then teamed up with Dennis Ritchie, the author of the first C compiler in 1973. They rewrote the UNIX kernel in C - this was a big step forwards in terms of the system's portability - and released the Fifth Edition of UNIX to universities in 1974
- The Seventh Edition, released in 1978, marked a split in UNIX development into two main branches: SYSV (System 5) and BSD (Berkeley Software Distribution)

The UNIX family



- Linux is a free open source UNIX OS for PCs that was originally developed in 1991 by Linus Torvalds, a Finnish undergraduate student. Linux is neither pure SYSV or pure BSD. Instead, incorporates some features from each (e.g. SYSV-style startup files but BSD-style file system layout) and aims to conform with a set of IEEE standards called POSIX (Portable Operating System Interface)
- The open source nature of Linux means that the source code for the Linux kernel is freely available so that anyone can add features and correct deficiencies
- As Linux has become more popular, several different development streams or distributions have emerged, e.g. Redhat, Slackware, Mandrake, Debian, and Caldera, UBUNTU



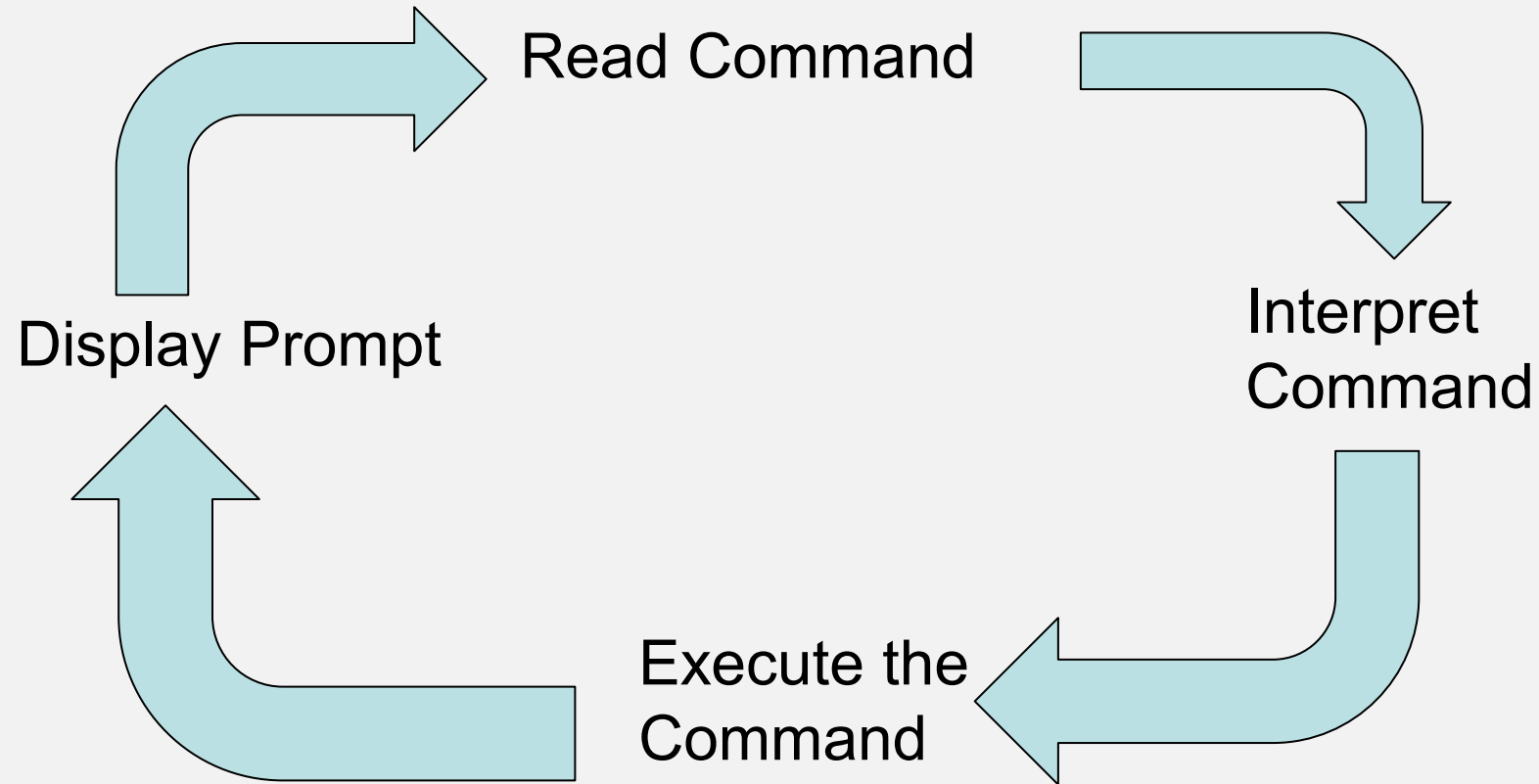
Command interpreter



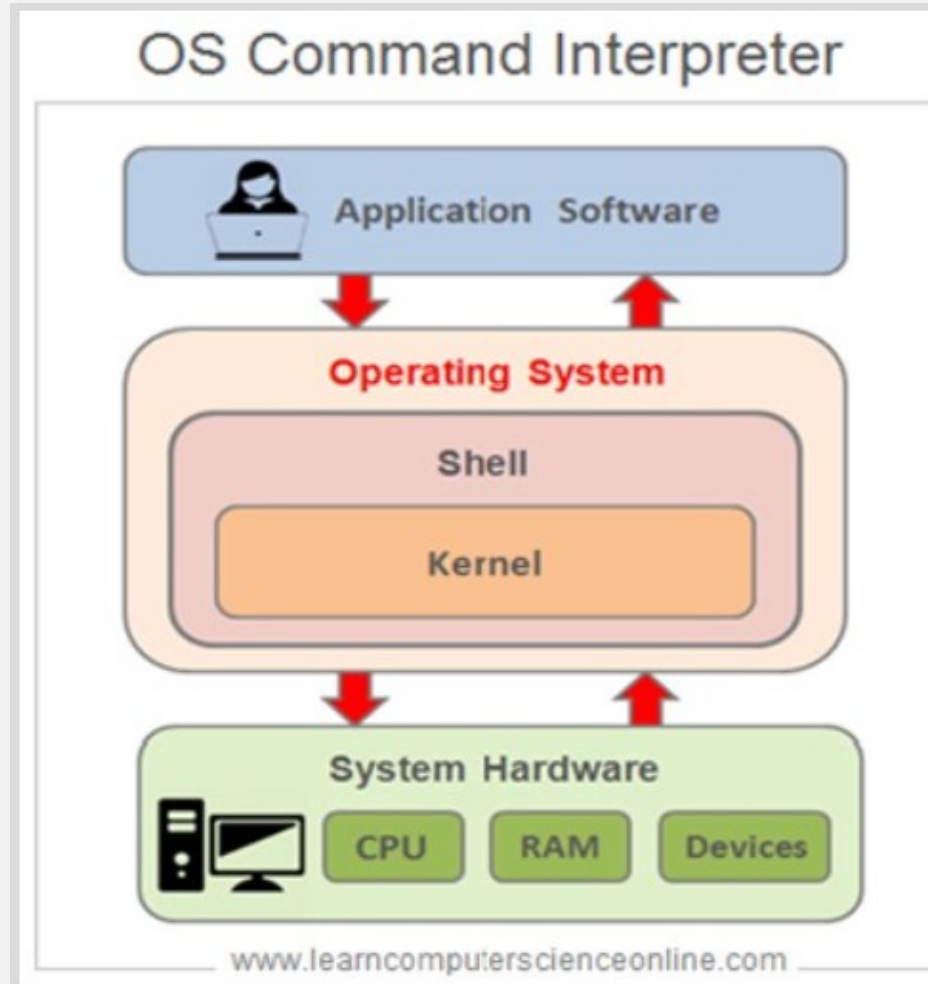
The Shell

- The shell is perhaps the most important program on the Unix system, from the end-user's standpoint. The shell is your interface with the Unix system, the middleman between you and the kernel
- The shell is a type of program called an interpreter. An interpreter operates in a simple loop: It accepts a command, interprets the command, executes the command, and then waits for another command
- The shell displays a "prompt," to notify you that it is ready to accept your command
- The shell recognizes a limited set of commands, and you must give commands to the shell in a way that it understands

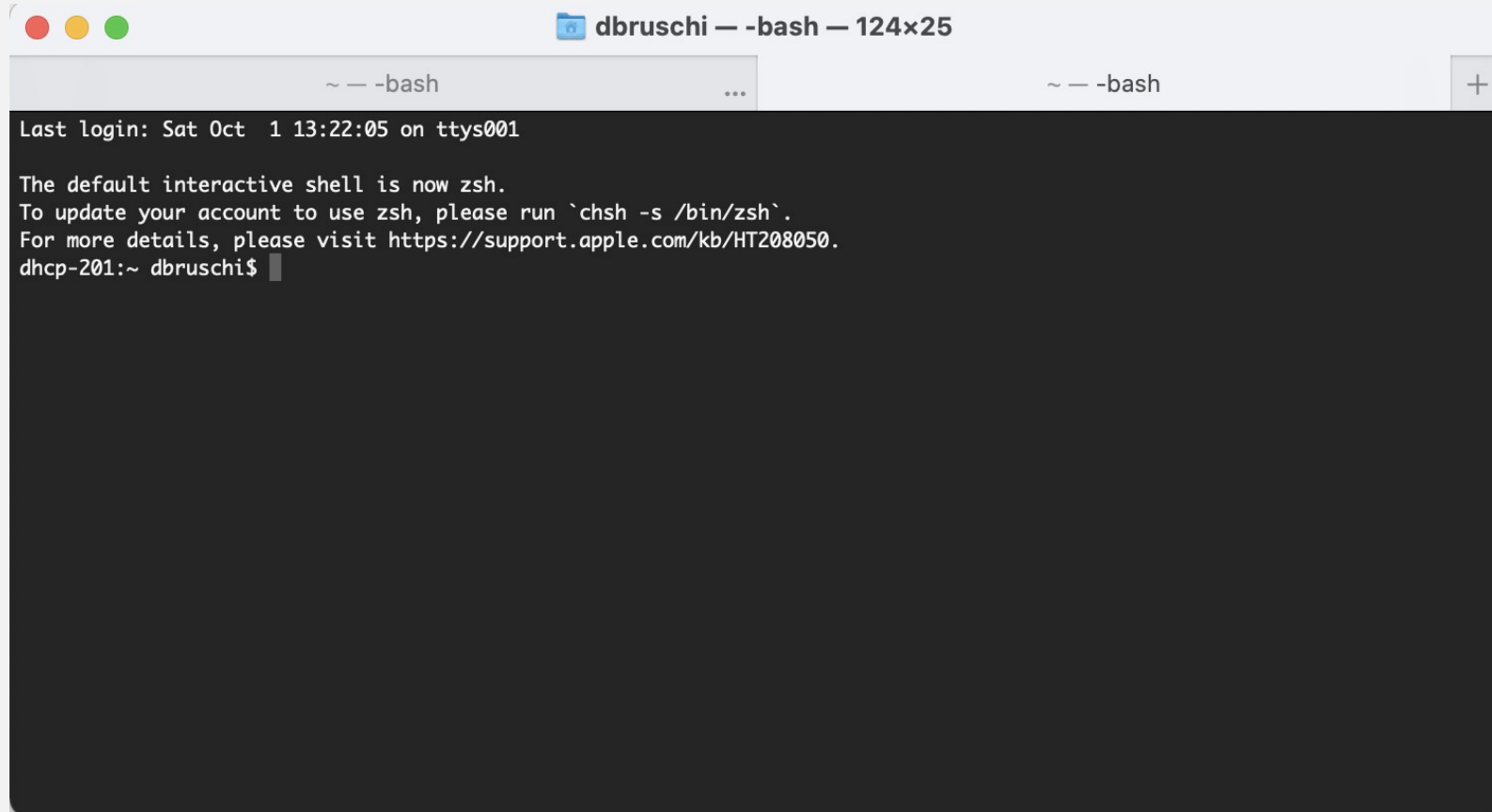
Shell



Shell interfaces



Unix shell



A terminal window titled "dbruschi — -bash — 124x25" is shown. The window contains the following text:

```
~ — -bash  
Last login: Sat Oct 1 13:22:05 on ttys001  
  
The default interactive shell is now zsh.  
To update your account to use zsh, please run `chsh -s /bin/zsh`.  
For more details, please visit https://support.apple.com/kb/HT208050.  
dhcp-201:~ dbruschi$
```

Logging to Linux

- When you connect to a UNIX computer remotely (using telnet) or when you log in locally using a text-only terminal, you will see the prompt:

```
login:
```

- At this prompt, type in your username and press the enter/return/ key. Remember that UNIX is case sensitive (i.e. Will, WILL and will are all different logins). You should then be prompted for your password:

```
login: will
```

```
password:
```

- Type your password in at the prompt and press the enter/return/ key. Note that your password will not be displayed on the screen as you type it in. If everything is ok you should be presented with a shell prompt which looks something like this:

```
$
```

- To log out of a text-based UNIX shell, type "exit" at the shell prompt (or if that doesn't work try "logout"; if that doesn't work press ctrl-d).

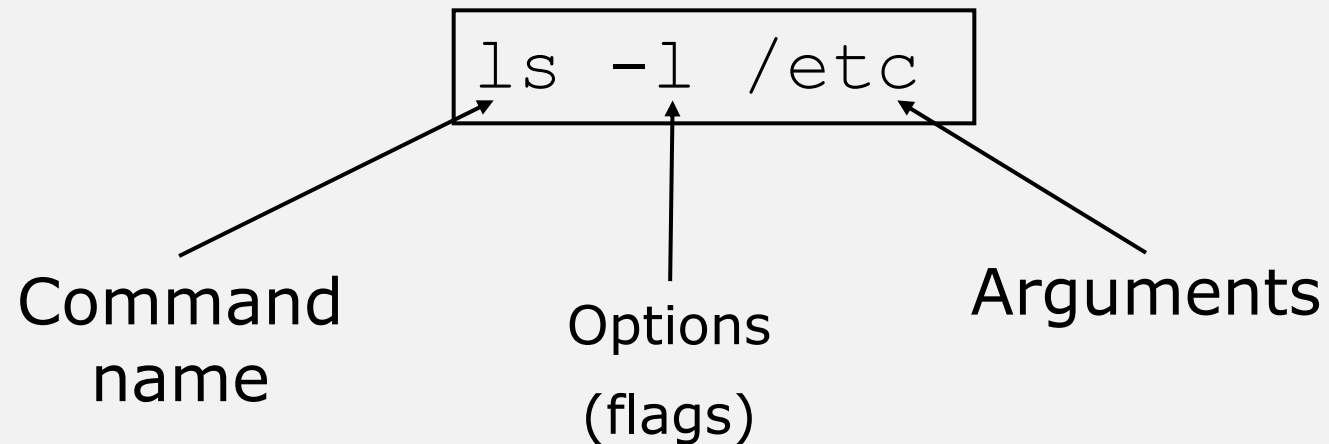
UNIX SHELL

- Once you have the prompt you can give commands to the operating system
- A UNIX command line consists of the name (the "command" is the name of a shell command, system utility, or application program) followed by its "arguments" (options and names of target files and/or expressions). The general syntax for a UNIX command is

\$ command [-options] [targets]

Linux Command Basics

- To run a command, type the name and arguments on the command line



Playing with the Shell 1

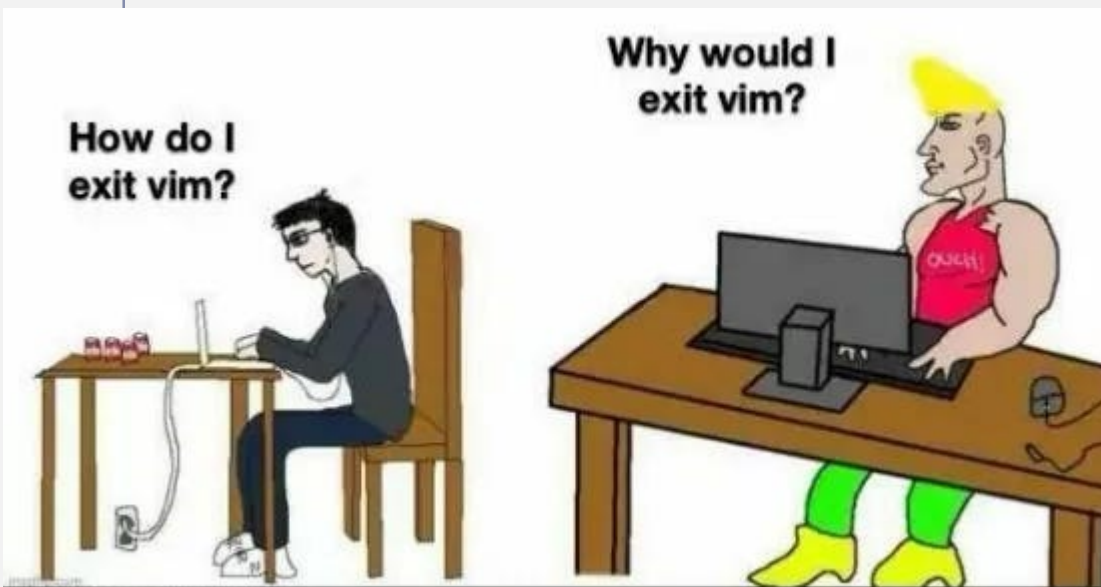
- Enter these commands at the unix prompt and try to interpret the output:
 - `echo hello world`
 - `date`
 - `hostname`
 - `arch`
 - `uname -a`
 - `dmesg | more` (you may need to press q to quit)
 - `uptime`
 - `whoami` (`/bin/bash` vs `/bin/sh`)
 - `w` / `who`
 - `id`

Playing with the Shell 2

- `finger (man vs --help)`
- `top (you may need to press q to quit)`
- `echo $SHELL`
- `yes please (you may need to press Ctrl-c to quit)`
- `time sleep 5`
- `history`
- `clear`
- `cal 2000`
- `cal 9 1752 (do you notice anything unusual?)`
- `bc -l (type quit or press Ctrl-d to quit)`
- `echo 5+4 | bc -l`
- `echo {con,pre}{sent,fer}{s,ed}`



Text editors – VI/VIM



ProgrammerHumor.io

SAY CAN'T QUIT VIM JOKE AGAIN



imgflip.com



> thecodinglove.com



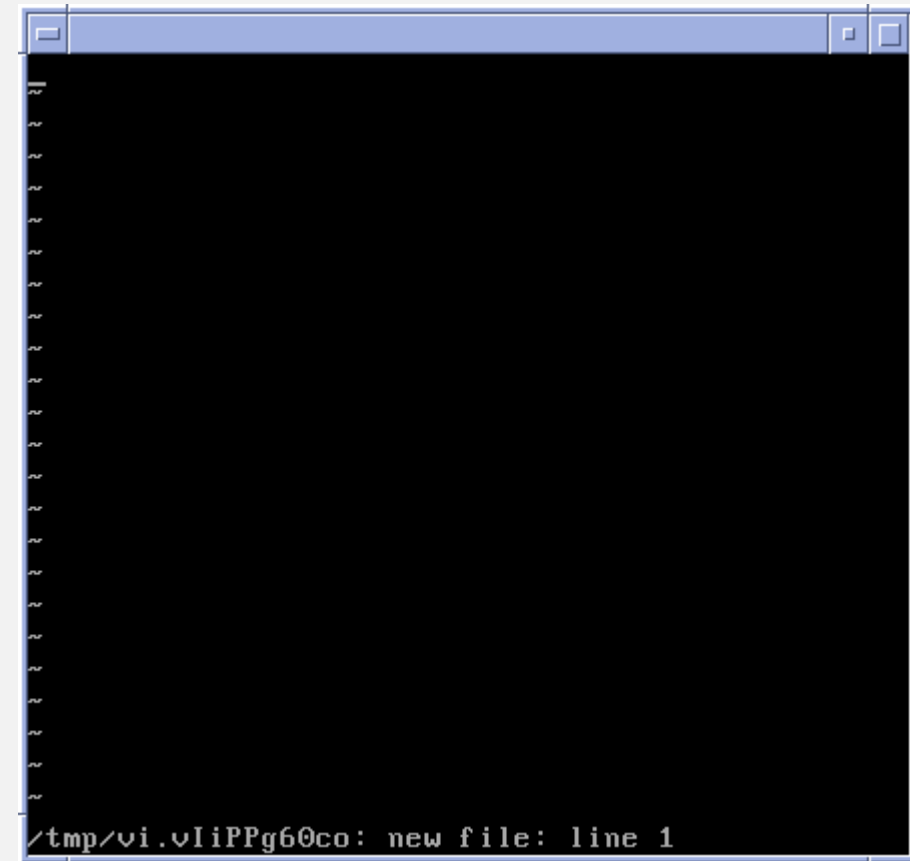
Text editors – VI/VIM

- Two working modes: insertion mode and command mode.

Editor begins in command mode: cursor movement, text deletion and pasting occur. Insertion mode begins upon entering an insertion or change command.

[ESC] returns the editor to command mode

- You can quit by typing :q



Text editors – nano

```
GNU nano 6.4 IMPROVEMENTS
Improvements in GNU nano
=====

Since 6.0:
- Option --zero hides the interface and uses the whole terminal for editing.
- Colors can be given also in #rgb hexadecimal, to select the nearest color
  from the 6x6x6 color-cube palette available on 256-color terminals.
- Fourteen new color names are available, from rosy to crimson.

Since 5.0:
- A search briefly spotlights the found text, in black on yellow by default.
- Option --minibar reduces the interface to a bottom bar with basic info.
- The cursor skips over combining characters, <Del> deletes them together
  with the character they combine with, but <Bsp> deletes them separately.
- For using libmagic the option --magic or -! or 'set magic' is required.
- With --stateflags the state of some things is shown in the title bar.
- M-Bsp deletes a word leftward (like in Bash).
- With --indicator a "scrollbar" is shown, indicating position+portion.
- M-Ins places an anchor, M-PgUp/M-PgDn jump to the nearest anchor.
- Toggling help lines (M-X) and Refresh (^L) work nearly everywhere.
- Colors can be modified with the attributes "bold," and/or "italic,".
- Nine new color names: from pink and purple to orange and latte.

^G Help      ^O Write Out  ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File  ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```





Unix Processes

- All UNIX processes are identified by a unique process identifier or PID
- An important process that is always present is the init process. This is the first process to be created when a UNIX system starts up and usually has a PID of 1
- All other processes are said to be "descendants" of init.

ps command

- To find out the processes associated to the underlying shell use **ps**
 - `$ ps`
 - | PID | TTY | TIME | CMD |
|-------|--------|----------|------|
| 17717 | pts/10 | 00:00:00 | bash |
| 27501 | pts/10 | 00:00:01 | find |
| 27502 | pts/10 | 00:00:00 | ps |
 - You can also use `ps` to show all processes running on the machine (not just the processes in your current shell):
 - `$ ps -fae / ps -aux`

Foreground - background

- Most shells provide sophisticated process control facilities that let you control running processes
- Jobs can either be in the foreground or the background.
- There can be only one job in the foreground at any time. The foreground job has control of the shell with which you interact - it receives input from the keyboard and sends output to the screen
- The foreground process may be suspended, i.e. temporarily stopped, by pressing the Ctrl-Z key. It can be restarted by pressing `"fg"`, `"bg"`
- Background jobs can also be run directly from the command line, by appending a '&' character to the command line:
 - `$ find / -print 1>output 2>errors &`

kill command

- To terminate a process abruptly, use the **kill** command followed by the process's PID
- `$ kill 23456` would terminate process 23456. Kill only sends the process a [signal](#) requesting it shutdown and exit gracefully (the SIGTERM signal), so this may not always work. To force a process to terminate abruptly (and with a higher probability of success), use a -9 option (the SIGKILL signal)
- `$ kill -9 23456`
- One other useful process control utility that can be found on most UNIX systems is the **pkill** command. You can use pkill to kill processes by name instead of PID.
- `$ pkill echo`

Pipe

- The pipe ('|') operator is used to create concurrently executing processes that pass data directly to one another
- `command 1 | command 2 | command 3 | .....`
- `$ ls -la | more`
- The output of the `ls` command is passed as input to the command `more`
- `$ cat zoo.text | grep "mammals"`
finds all lines in `zoo.txt` that contain the string "mammals"



I/O Redirection

- The output from programs is usually written to the screen, while their input usually comes from the keyboard (if no file arguments are given). In technical terms, we say that processes usually write to **standard output** (the screen) and take their input from **standard input** (the keyboard).
- There is in fact another output channel called **standard error**, where processes write their error messages; by default, error messages are also sent to the screen.
- In UNIX the file descriptors 0, 1 and 2 are assigned to standard input, standard output and standard error respectively

Output Redirection

- To redirect standard output to a file instead of the screen, we use the **>** operator:
 - `$ echo hello`
 - `hello` (output sul video)
 - `$ echo hello > output`
 - `$ cat output`
 - `Hello`
- In this case, the contents of the file output will be destroyed if the file already exists. If instead we want to append the output of the echo command to the file, we can use the **>>** operator

Error redirection

- To capture standard error, prefix the `>` operator with a `2`
 - `$ cat File 2>errors`
 - `$ cat errors`
 - `cat: nonexistent: No such file or directory`
- You can redirect standard error and standard output to two different files:
 - `$ find . -print 1>files 2>errors`
- or to the same file:
 - `$ find . -print 1>output 2>output`

Input redirection

- Standard input can also be redirected using the `<` operator, so that input is read from a file instead of the keyboard
 - `$ cat < output`
- You can combine input redirection with output redirection but be careful not to use the same filename in both places. For example:
 - `$ cat < output > output`
- The Previous command will destroy the contents of the file `output`. This is because the first thing the shell does when it sees the `>` operator is to create an empty file ready for the output.



Environment Variables

Environment

- “An environment variable is a named object that contains data used by one or more applications” [1](#)
- Set of key-value pairs that can be read/modified by a program in execution and propagated to child processes

```
SHELL=/bin/bash
LANGUAGE=en_US.UTF-8
PWD=/root
HOME=/root
PATH=/root/.cargo/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/snap/bin:/root/.dotnet/tools:/root/minio-binaries/
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/0/bus
```



Environment

- Part of the operating environment in which a process runs
- `env` / `export` commands to view/modify the environment
- To persistently set environment variables:
 - `/etc/environment`
 - `~/.profile`
- Security question: can they be dangerous in any way?
 - Can we exploit a `suid` binary?
 - CVE-2021-4034

PATH variable exploitation

- “PATH is an environmental variable in Linux and other Unix-like operating systems that **tells the shell** which directories to **search for executable files**”[1](#)
- What happens if the PATH variable is not correctly set?[2,3](#)

```
PATH=:/bin/:/usr/bin
```

- There is an empty path at the beginning, meaning that **our shell** will first search for binaries in the current directory
 - Not good for poorly written setuid binaries, if you can find one

CVE-2021-4034 in a nutshell



Accessing the environment

```
#include <stdio.h>
void main(int argc, char* argv[], char* envp) {
    int i = 0;
    while(envp[i] != NULL) {
        printf("%s\n", envp[i++]);
    }
}
```

- In a C program, environment variables can be accessed through the third argument of the main function()
- OS-specific syscalls, like [getenv\(\)](#)
- External `environ` variable

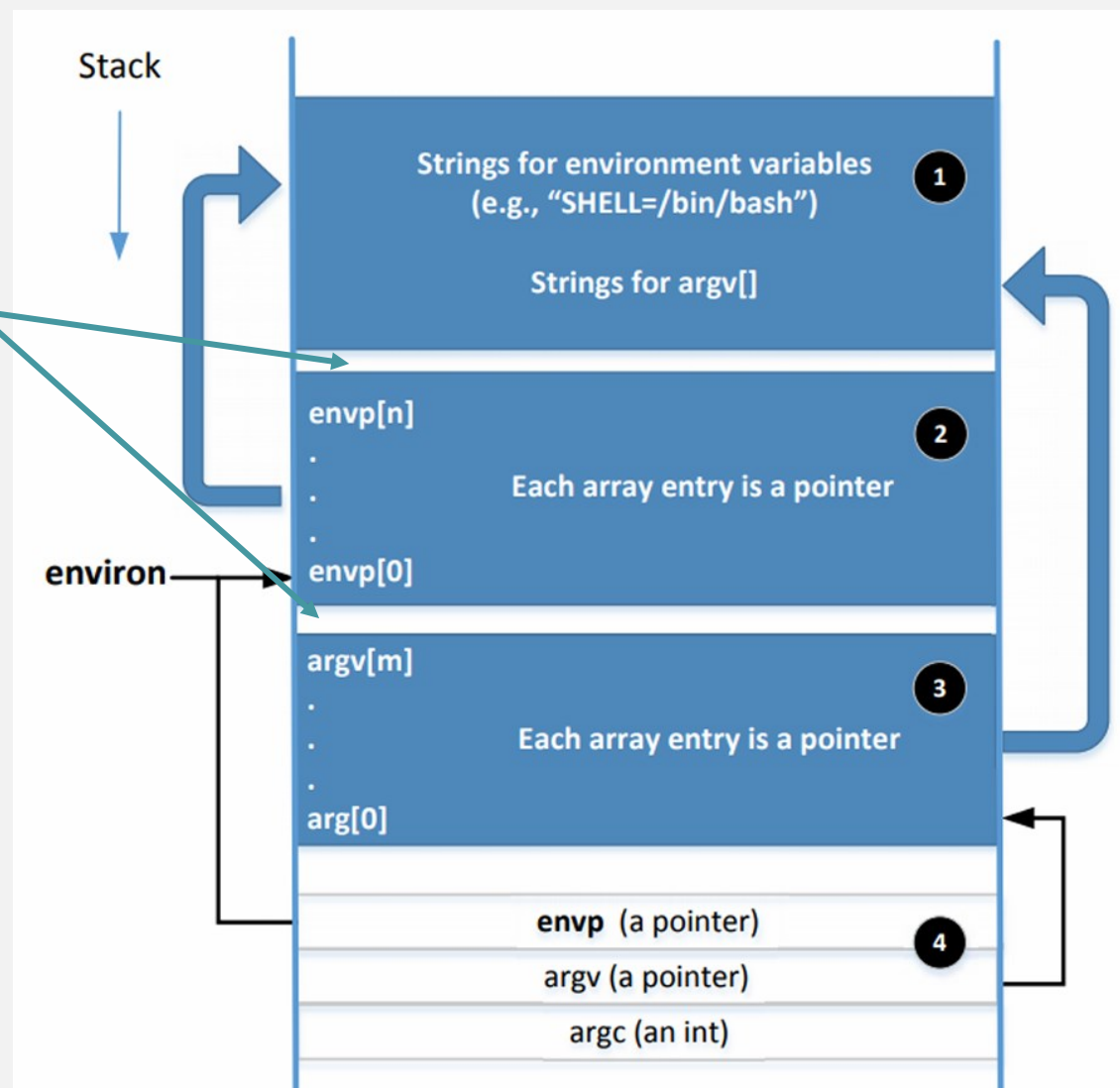
Environment for child processes

```
int execve(const char *path, char *const _Nullable argv[],  
           char *const _Nullable envp[]);
```

- The [execve\(\)](#) syscall receives as input a pointer to an environment and exposes it to the new process
- Same for the arguments of our application
- Path must be a valid and existing path to an executable, execve will not search for binaries

Environment memory layout

NULL pointers to
indicate the end of
the list



CVE-2021-4034 in a nutshell

- Local Privilege Escalation (from any user to root) in polkit's pkexec, a SUID-root program that is installed by default on every major Linux distribution
- pkexec - Execute a command as another user
- Linux strips unsafe^{[1](#),[2](#)} environment variables before invoking setuid programs, so pkexec's environment **shouldn't** be vulnerable to tampering.

CVE-2021-4034 in a nutshell

```
435 main (int argc, char *argv[])
436 {
...
534     for (n = 1; n < argc; n++)
535     {
...
568     }
...
610     path = g_strdup (argv[n]);
...
629     if (path[0] != '/')
630     {
...
632         s = g_find_program_in_path (path);
...
639         argv[n] = path = s;
640     }
```

argv[0] can be a NULL entry,
implying **argc** == 0

If **argc** == 0, **n** will be 1

argv[1] overlaps with **envp[0]**

No problem here, let's assume a
safe PATH

We can overwrite **envp[0]**



CVE-2021-4034 in a nutshell

- Using the buffer overflow on **argv**, the unsecure variable **GCONV_PATH** can be reintroduced into the environment
- With **GCONV_PATH** again in the environment, pkexec can be tricked to load a shared library from a path under the attacker's control
- We have obtained code execution in the root user context



C/C++/Assembly development environment

A quick example



Prerequisites

- We will write some C/Assembly code in the next lessons. We need a development environment
- `sudo apt update && sudo apt-get install build-essential
strace ltrace gdb`

First C program

```
#include <stdio.h>
void main(int argc, char* argv[], char* envp[])
{
    int i = 0;
    while (envp[i] != NULL)
    {
        printf("%s\n", envp[i++]);
    }
}
```

- Just prints the environment
- Compile with `gcc envp.c -o envp`

strace & ltrace debugging tools

- `strace` runs a command until it exits, intercepting the system calls made by the process
The name of each system call, its arguments, and its return value are printed to standard error
- `ltrace` is like `strace`, but intercept calls to shared libraries imported by the observed process



gdb - GNU Project debugger

- Classic Linux debugger to pause and inspect a program during execution
- Important for evaluating the correctness of programs you develop throughout the course.
- Let's check if what we said about the environment layout in memory is correct